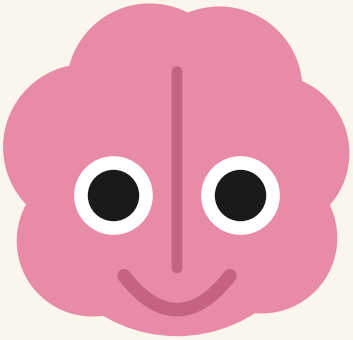


BrainRot Tracker

Project Spec Sheet

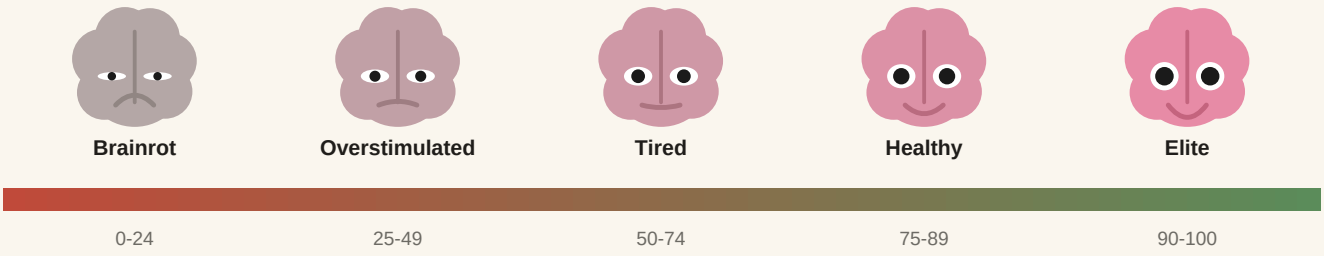
Every feature, the technology behind it, and the Android capability that makes it possible — explained for readers with no app-development background.

Android · Kotlin · Jetpack Compose · Room · Firebase

June 2026

1 · What the app is

BrainRot Tracker is an Android app that watches how many short videos (Instagram Reels, YouTube Shorts, TikTok videos, Snapchat Spotlight) you scroll through each day, shows you the damage in a friendly way — a cartoon brain that “rots” as you scroll — and helps you stop: daily limits, a blocking screen with three strictness modes, streaks, notifications, a floating live counter, and a home-screen widget. Optionally, you can sign in with Google to back up your daily totals to the cloud.



The brain mascot across the health spectrum. The score (0–100) is recomputed all day from your counts and screen time vs. your limits; the face is drawn entirely in code.

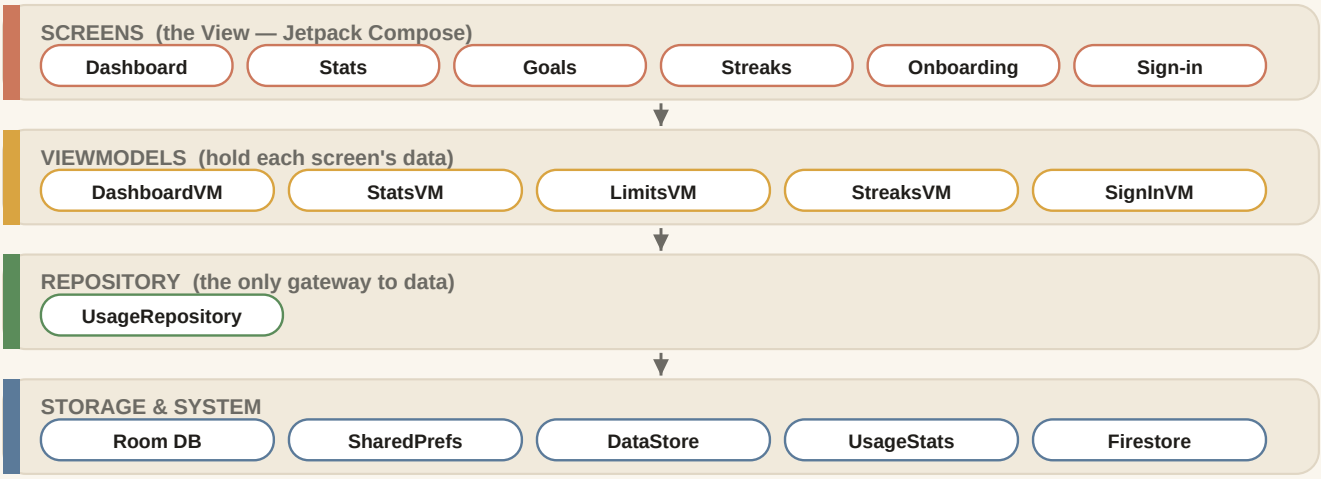
2 · The technology stack

The tools the app is built with, in plain words:

Technology	What it is
Kotlin	The programming language — the “English” the app's instructions are written in. Google's recommended language for Android.
Jetpack Compose	The toolkit that draws every screen. You describe the UI in code (“a column with a title, then a slider…”) and it redraws automatically whenever data changes.
Gradle	The build system: assembles code, images and libraries into the installable APK file.
Android SDK	Android's own capabilities (notifications, databases, overlays…). The app runs on Android 7.0+ and targets Android 16.
Room	Library managing the local database (§8).
Firebase	Google's cloud service for sign-in and the optional backup (§7).

How the code is organized (MVVM)

Each screen is intentionally “dumb” — it only displays data. A **ViewModel** feeds it that data and survives rotations; ViewModels talk to one **Repository**, the single gatekeeper to storage. UI code never touches the database directly, which keeps bugs contained.



The four layers. Arrows show the only allowed direction of dependency.

3 · Counting reels — the core trick

The app counts videos you swipe *inside other companies' apps*. The Android capability that makes this possible is the **Accessibility Service** — a feature built for screen readers used by blind users. Once you enable it in system settings, Android streams the service events like "a list scrolled," plus a machine-readable outline of what's on screen. The service is locked to the four social apps in its configuration, so Android sends it nothing about any other app.

The counting pipeline, fired on every swipe. Code: service/ReelCounterService.kt

How it tells a reel from a normal feed

A reels player is a **full-screen list showing exactly one item at a time** (a "pager"); a normal feed shows many items at once, so feeds never trigger counts. Each platform needs its own detection strategy:

Platform	Strategy
Instagram, Snapchat	Watch the pager's *current item index*; index goes up = you swiped to the next reel.
YouTube Shorts	No index exposed, so the app builds a "signature" (creator handle + title + button labels) and counts when it changes to one not seen recently — rewatching doesn't double-count.
TikTok	The whole app is short videos, so every forward scroll counts.

Two safety valves prevent over-counting: a **debounce** (counts on the same platform must be ≥1.2 s apart) and a **rate limit** (the screen outline is inspected at most every 0.8 s, because walking it is expensive). Confirmed counts are written to the local database on a background thread (Kotlin **coroutines**) so the phone never stutters.

4 · The floating counter bubble

While you scroll, a small pill floats *on top of* Instagram/TikTok showing the brain face and today's count; tap for a per-platform breakdown, drag to move. This uses Android's **"Display over other apps"** permission (SYSTEM_ALERT_WINDOW) — the same mechanism as Messenger's old chat heads.

It lives in a **foreground service** (code that keeps running with no screen open; Android requires the permanent “tracking active” notification for this). The brain face and the platform logos are **drawn by hand on a Canvas** — pure geometry, no image files, so no trademarked assets ship in the app. A 1-second **heartbeat** polls which app is in front: the accessibility service hears nothing from apps outside its list, so polling is the only way to notice you've left and hide the pill. Code: service/FloatingCounterService.kt

5 · App blocking — three modes

When a platform hits its daily limit, a full-screen dark overlay covers it. **evaluateBlocking()** is the single gatekeeper and runs at three moments: every counted reel, every time a tracked app comes to the foreground, and every heartbeat tick — so reopening the app after dismissing the overlay re-triggers the block. “Close app” uses a superpower only accessibility services have: programmatically pressing the Home button. At midnight the day-rollover (§6) resets counts, lifting all blocks.

HARD	SNOOZE	REMIND
Blocked until midnight	*5 more minutes, then back*	*Once per app visit*
Overlay returns EVERY time you reopen the app. Only button: "Close app" (sends you to home screen)	Dismiss = 5-minute grace. Deadline saved to disk, so a restart can't cheat it. Buttons: Close app / Snooze	Shows once each time the app is opened; "Got it" lets you keep scrolling this visit. Buttons: Close app / Got it

The three user-selectable modes (Settings → App Blocking).

6 · Screen time, streaks & day rollover

Screen-time minutes

Android keeps a system-wide usage log (what Digital Wellbeing shows). After you grant **Usage access**, the app reads the **raw event log** — every “app came to front / left front” timestamp — and replays it like a stopwatch for exact midnight-to-midnight minutes. (Android's pre-aggregated totals API leaks time across day boundaries, so the app deliberately avoids it.) Minutes are never stored; they're recomputed live. Code: data/util/ScreenTimeHelper.kt

Streaks — the “lazy catch-up” design

Every day you stay under your limits earns a streak day, shown on a calendar. Phones kill scheduled background jobs aggressively, so instead of a midnight alarm the app evaluates streaks **opportunistically**: whenever the service starts, the app opens, or the first scroll after midnight happens, it walks every not-yet-judged day, compares counts to limits, and writes the verdicts. A day with no activity at all counts as a win. A missed midnight just gets caught up at the next opportunity.

Day rollover

On every event the service cheaply compares “what day is it now” with “what day are my counters for.” On the first event after midnight it: resets counters (lifting blocks), evaluates yesterday's streak, fires the daily-summary notification, re-arms milestone alerts, and uploads to the cloud if enabled.

Notifications

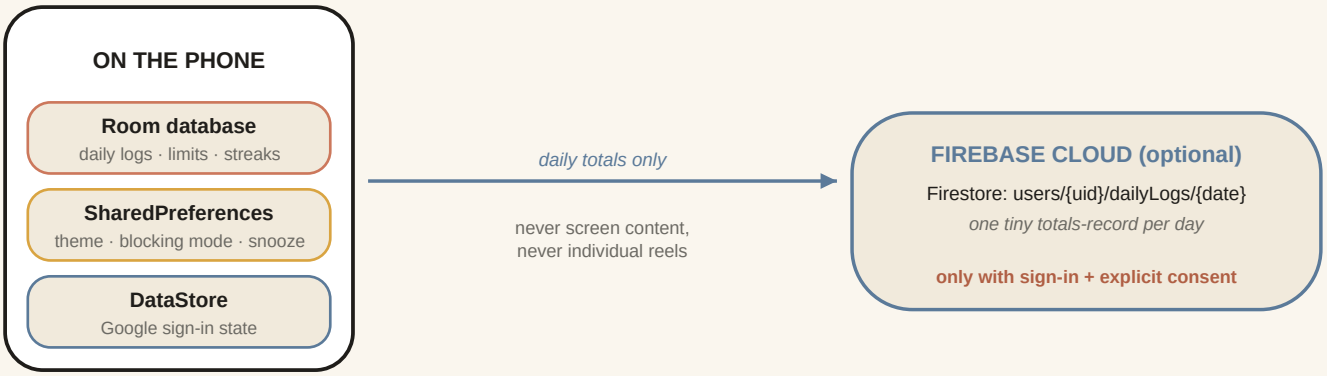
Three channels (Android's mute-one-kind-not-all system): **milestones** (“50 reels! Time for a break?” at 25/50/75/100/150/200), the **daily summary** recap, and the required **service notice**. On Android 13+ the app fires the real permission dialog during onboarding. Code: notification/NotificationHelper.kt

Home-screen widget

A glanceable widget (today's count + health) built with **Glance**, Google's library for writing widgets in Compose-style code. It reads the database directly and refreshes when Android asks — a snapshot, not live. Code: widget/BrainRotWidget.kt

7 · Google sign-in & cloud backup (optional)

The app is fully functional anonymously. Signing in lets it upload **one small record per finished day** — counts, minutes, health, limits, streak status — to your private cloud space. Three pieces: **Credential Manager** (Android's modern Google sign-in sheet) returns an ID token; **Firebase Authentication** exchanges it for a stable user id; **Cloud Firestore** (a cloud document database) stores the records. Writes are fire-and-forget — Firestore queues them offline and retries automatically.



What leaves the phone — and what never does. Code: data/sync/UsageSyncManager.kt

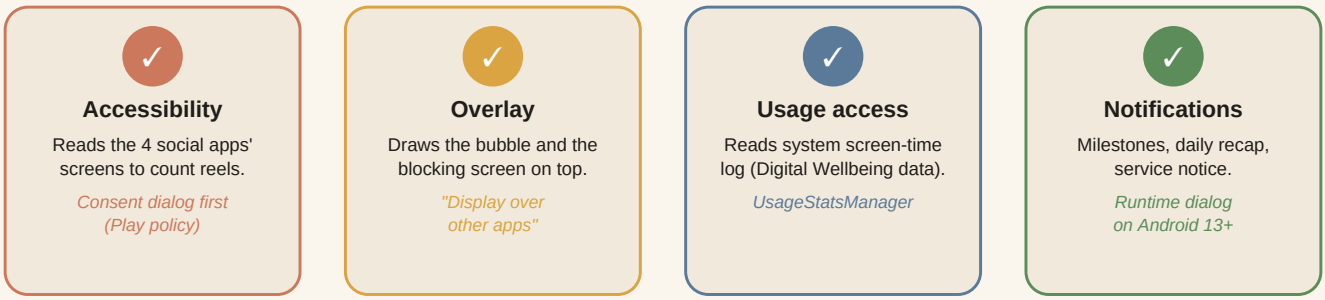
Because the counts come from the Accessibility API, Google Play scrutinizes any upload hard: it therefore requires sign-in **plus** an explicit consent dialog, sends only daily totals, and a Settings toggle turns it off anytime. The whole feature degrades gracefully — the project builds and runs even before the Firebase project exists.

8 · How data is stored on the phone

Store	What it is	What lives there
Room database	Wraps SQLite, the tiny database engine inside every Android phone. Tables are defined as Kotlin classes; reads are “Flows” — live pipes that re-emit when data changes, so screens update instantly.	DailyLog (counts + health per day), UserLimits (per platform), StreakRecord (verdict per day)
SharedPreferences	Simple key-value file services can read instantly.	Theme, blocking on/off + mode, snooze deadlines, HUD size, consents
DataStore	The modern key-value store, friendly to background code.	Google sign-in state (email, name, photo)

9 · Onboarding & the four special permissions

A first-launch screen walks you through granting the permissions, with live GRANTED/PENDING badges that flip the moment you return from system settings. Everything is skippable, and onboarding is bypassed on later launches if the accessibility service is already on. Before opening accessibility settings, a **prominent-disclosure consent dialog** (a Google Play policy requirement) explains exactly what is read and that it stays on-device unless backup is enabled.



The four special grants. None use the normal permission popup except notifications.

10 · Screens, navigation & theming

Six screens — Dashboard, Stats, Goals (settings), Streaks, plus full-screen Onboarding and Sign-in flows that hide the bottom tab bar. Navigation uses **Navigation 3**, Google's Compose-native library: each destination is a tiny key object and a single back-stack list decides what's showing. Theming is System/Light/Dark over a warm cream-and-terracotta palette; the choice is held as live Compose state (instant recolor) backed by saved preferences (survives restarts), and the overlay bubble reads the same preference so it always matches.

11 · Release engineering (making it shippable)

Item	What it does
R8 / ProGuard	Release builds strip unused code, shrink resources, scramble names, and delete debug logging — smaller and harder to reverse-engineer.
Desugaring	Rewrites modern date/time code at build time so it runs on Android 7, the minimum version.
Signing	Apps must be cryptographically signed; credentials live in an untracked keystore.properties file.
Hardening	Overlay service not reachable by other apps; unused permissions removed; full accessibility disclosure text.
Still manual	Confirm the permanent app id, create the Firebase project + security rules, generate the signing key, host a privacy-policy URL disclosing the optional upload.

12 · Where to find things

Path	Contents
service/ReelCounterService.kt	Accessibility service: counting, blocking trigger, day rollover
service/FloatingCounterService.kt	Overlay: bubble, popup, blocking scrim, Canvas brain
data/repository/UsageRepository.kt	All data logic: health score, streak evaluation
data/util/ScreenTimeHelper.kt	Usage-stats event-replay stopwatch
data/sync/UsageSyncManager.kt	Firestore daily-aggregate upload
ui/screens/…	dashboard / stats / limits / streaks / onboarding / signin
notification/ · widget/ · theme/	Notification channels · Glance widget · colors + ThemeController
Navigation.kt · NavigationKeys.kt	Back stack, bottom bar, screen keys